

Universidad Nacional
Sección Regional Huetar Norte y Caribe,
Campus Sarapiquí.

Curso:

Diseño y Programación de Plataformas Móviles.

Informe de investigación React Native

Docente:

Mag. Rachel Bolívar Morales

Estudiantes:

Deibys Gutiérrez Morales

Yeimer Requene Madrigal

Ciany Amador Solís

Yeiler Madrigal

Arturo Chavarría

I Ciclo 2026

Fecha de entrega:

06 de junio 2026

1. Historia y origen del framework

El origen de React Native se remonta a uno de los mayores desafíos estratégicos de Facebook (ahora Meta). Antes de la llegada de React Native, el desarrollo móvil obligaba a los equipos de ingeniería a bifurcar sus proyectos: debían programar apps completamente independientes para Android y para iOS. Esto implicaba dominar ecosistemas totalmente separados, utilizando lenguajes como Java o Kotlin en el entorno de Google, y Objective-C o Swift para el de Apple.

Este panorama cambió a partir de 2013, cuando el ingeniero Jordan Walke aprovechó un hackathon interno en Facebook para explorar una alternativa diferente. Su propuesta consistía en renderizar componentes visuales nativos de iOS controlados desde un hilo de JavaScript que operaba en segundo plano (Konicek, 2016). El éxito de este experimento abrió las puertas para fusionar el paradigma declarativo de React —que ya triunfaba en el desarrollo web de la época— con la fluidez y velocidad de las plataformas móviles.

La consolidación del proyecto llegó a inicios de 2015, tras presentarse formalmente en la React.js Conf. En marzo de ese año se liberó el código abierto enfocado en iOS, sirviendo como base para que en septiembre de 2015 se expandiera el soporte hacia Android. A partir de ese hito, el framework transformó el diseño de software móvil (Konicek, 2016), demostrando que las empresas podían centralizar sus operaciones en un único repositorio de código compartido sin que la aplicación perdiera su rendimiento táctil ni la experiencia nativa del dispositivo. Esta situación dejaba mayores costos de desarrollo, mantenimiento y actualización. Facebook buscó una solución que permitiera compartir gran parte del código entre plataformas manteniendo una experiencia de usuario similar a la de una aplicación nativa (Konicek, 2016).

A lo largo de la década de 2020, el framework experimentó una maduración institucional. Dejó de ser visto como una herramienta exclusiva para interfaces sencillas para convertirse en una infraestructura robusta de nivel empresarial, impulsada no solo por Meta, sino por un comité directivo global conformado por corporaciones como Microsoft y Amazon (Meta, 2025).

2. Arquitectura y funcionamiento

Para entender el funcionamiento de React Native, es necesario analizar la transición tecnológica que ha definido al framework en los últimos años: la coexistencia y migración de la Arquitectura Heredada (Bridge Architecture) hacia la Nueva Arquitectura.

2.1 La Arquitectura del Puente (Bridge) - Modelo Heredado

Originalmente, React Native operaba dividiendo la aplicación en tres hilos (threads) principales interconectados por un elemento central llamado el Bridge (Liu, y otros, 2023) :

El funcionamiento interno de React Native se sostiene sobre una división de tareas ejecutada principalmente por tres hilos imperativos. El primero de ellos es el Hilo de JavaScript, el cual opera como el cerebro lógico de la aplicación; desde este entorno se procesa todo el código de negocio, se administran las peticiones HTTP y se gestiona el ciclo de vida de los componentes.

Sin embargo, para que el usuario perciba una aplicación fluida, este hilo debe comunicarse constantemente con el Hilo Nativo. Esta es la línea principal del sistema operativo anfitrión (iOS o Android) y su única tarea es procesar las interacciones directas del usuario y renderizar los elementos gráficos en pantalla.

El puente entre la lógica web y la interfaz móvil lo complementa el Hilo de Sombra. Este último componente actúa como un calculador geométrico intermedio: toma los diseños estructurados mediante Flexbox en el código de JavaScript y, utilizando el motor de Meta, los traduce a coordenadas físicas y dimensiones exactas que las plataformas nativas (Liu, y otros, 2023) puedan interpretar y dibujar correctamente.

El Bridge actuaba como un canal de comunicación asíncrono. Cada vez que el hilo de JS requería un cambio visual, serializaba los datos en formato JSON, los enviaba a través del puente, y el hilo nativo los deserializaba para ejecutarlos. (Liu, y otros, 2023)

Aunque para esto también hay una limitación crítica ya que este proceso asíncrono generaba cuellos de botella severos cuando se procesan eventos de alta velocidad en tiempo real, como listas masivas con scroll rápido o animaciones complejas, ya que los datos se acumulaban en el puente.

2.2 La Nueva Arquitectura

Para poder superar estas limitaciones, React Native introdujo una nueva arquitectura que está basada en tres tecnologías:

JavaScript Interface (JSI)

El JSI permite que el código JavaScript se pueda comunicar directamente con el código nativo sin la necesidad de utilizar el Bridge tradicional, lo cual va a reducir considerablemente la sobrecarga de comunicación y mejora el rendimiento de la aplicación. (Chisti, 2025)

Fabric

Es el nuevo sistema de renderizado de React Native. Al apoyarse directamente sobre JSI, Fabric logra que el renderizado de la interfaz de usuario ocurra de manera síncrona. Esto cambia radicalmente la experiencia, ya que el sistema prioriza las respuestas inmediatas ante gestos o toques en la pantalla, garantizando transacciones fluidas y alineando con precisión milimétrica la lógica interna del software con los cambios visuales (Chisti, 2025).

Turbo Modules

Por otro lado, la arquitectura de TurboModules renueva por completo el esquema tradicional de integración con el hardware. La ventaja de este componente radica en su capacidad de carga bajo demanda (lazy loading); de este modo, los módulos nativos se ejecutan únicamente cuando la aplicación los solicita de forma explícita, disminuyendo el consumo innecesario de memoria RAM y acelerando drásticamente el arranque inicial del sistema.

Flujo de funcionamiento

El funcionamiento general de React Native puede resumirse de la siguiente manera:

1. El usuario interactúa con la interfaz móvil.
2. Los eventos son procesados por el código JavaScript.
3. React actualiza el estado de la aplicación.
4. React Native sincroniza los cambios con los componentes nativos.
5. El sistema operativo renderiza los elementos visuales correspondientes.

Gracias a este modelo, los desarrolladores pueden trabajar utilizando componentes reutilizables y una única lógica de negocio compartida entre Android e iOS. (Meta, 2025)

3. Lenguajes utilizados

La gran ventaja operativa de React Native radica en su capacidad para combinar distintos entornos de programación según los requerimientos técnicos del proyecto. Este acoplamiento estratégico busca equilibrar la rapidez metodológica del desarrollo web con la potencia y optimización de bajo nivel que exigen los sistemas operativos de los teléfonos actuales (Meta, 2025). Dentro de este esquema se encuentran:

- **JavaScript:** es el que asume el rol de eje central, siendo la herramienta con la que los ingenieros estructuran las reglas de negocio, definen las rutas de navegación y diseñan la arquitectura declarativa de las pantallas. Se utiliza bajo el paradigma declarativo de React Native.
- **TypeScript:** Aunque inicialmente era opcional, en los últimos años se ha convertido en el estándar nativo por defecto al inicializar proyectos en React Native. Aporta tipado estático, reduciendo drásticamente los errores en tiempo de ejecución en aplicaciones a gran escala y facilitando la documentación del código compartido. (Meta, 2025)
- **Lenguajes nativos:** A pesar de que el framework resuelve el grueso del desarrollo mediante JavaScript o TypeScript, ciertos escenarios exigen interactuar directamente con el hardware del teléfono. En el ecosistema

Android, por ejemplo, el uso de Java o Kotlin se vuelve indispensable cuando se requiere programar módulos personalizados a la medida o explotar características avanzadas de las API de Google que no están expuestas de forma predeterminada en el entorno híbrido. En iOS pueden utilizarse Objective-C y Swift, estos lenguajes permiten integrar las características exclusivas de Apple y optimizar determinadas funcionalidades (Liu, y otros, 2023).

El acoplamiento técnico con las plataformas de bajo nivel se consolida a través de los denominados Native Modules y Native Components. Dichos mecanismos actúan como puentes de comunicación que permiten invocar subrutinas en Kotlin, Java, Swift u Objective-C directamente desde el entorno de JavaScript. Esta estrategia es crucial cuando el sistema necesita interactuar de forma directa con recursos del hardware que requieren permisos especiales o APIs específicas, tales como lectores biométricos, la cámara del dispositivo, herramientas de geolocalización o la gestión compleja de notificaciones push. Gracias a esta arquitectura, React Native combina la reutilización de código multiplataforma con el acceso completo a las capacidades nativas de Android e iOS (Meta, 2025)

4. Casos reales de uso

La viabilidad comercial e industrial de React Native queda demostrada por su adopción en plataformas globales con cientos de millones de usuarios activos diarios. A continuación, se analizan los casos más emblemáticos de su implementación estratégica:

<i>Empresa</i>	<i>Aplicación</i>	<i>Beneficios obtenidos</i>
<i>Facebook</i>	Facebook App	Reutilización de código y desarrollo más rápido
<i>Instagram</i>	Instagram	Menor tiempo de mantenimiento entre plataformas

<i>Discord</i>	Discord Mobile	Actualizaciones simultáneas para Android e iOS
<i>Microsoft</i>	Microsoft Teams y React Native Windows	Expansión multiplataforma
<i>Coinbase</i>	Coinbase Mobile	Menor costo de desarrollo y mantenimiento

En la tabla podemos ver que la adopción de React Native por organizaciones de gran escala demuestra que el framework es capaz de soportar aplicaciones con millones de usuarios activos. Las empresas obtienen beneficios importantes como la reducción de costos de desarrollo, la reutilización de una gran parte del código entre plataformas (Meta, 2025), la aceleración en la implementación de nuevas funcionalidades y una disminución en el esfuerzo de mantenimiento. Estas ventajas nos explican por qué React Native continúa siendo una de las principales alternativas para el desarrollo móvil multiplataforma en 2026.

Comparación Técnica con Android Nativo

Para que se pueda entender el enfoque que tiene cada uno de los enfoques primero hay que analizar cómo es que estos se comunican en lo que el programador realiza en código y el sistema operativo.

Enfoque de Android Nativo

Cuando se está desarrollando en Android native que será utilizando ya sea kotlin o Java, lo que pasa es que el código que está haciendo tiene que compilarse en bytes y que este lo ejecuta una máquina virtual que sería el ART- Android Runtime. Cuando las aplicaciones se realizan así no hay intermediarios entonces la aplicación tiene el acceso directo a todas las APIS del sistema operativo que serían como por ejemplo la cámara, bluetooth, almacenamiento y entre otros más, además se obtiene el acceso directo a los componentes de la interfaz del usuario.

Para poder manejar el renderizado visual, se tiene que utilizar un sistema tradicional de Android o mediante Jetpack Compose que se usa para poder dibujar directamente en la pantalla con el rendimiento máximo que ofrece puede ofrecer el dispositivo.

Enfoque de React Native

Este enfoque que fue creado por Meta funciona totalmente diferente al enfoque de Android native ya que este no tiene que traducir el código de JavaScript a Kotlin ya que lo que hace es ejecutar un motor de JavaScript dentro de la misma aplicación, el motor que se está utilizando es Hermes que es un motor que ya está optimizado para móviles.

EL reto más grande que tiene este enfoque es la comunicación porque históricamente react native usaba una arquitectura que se basaba en un puente, entonces esto es como que nos imaginemos que Android nativo y react son dos islas y que para tener una comunicación se tenía que hacer un paquete con la información pero en formato Json, entonces se envía de forma asíncronica y cuando ya se encuentra en el otro lado del puente se tiene que desempaquetar y una vez echo eso ya ahí si se pude hacer la ejecución de la acción pero esto hacía que se presentara con lentitud el proceso. (Bonaventure, 2023) (Soni, 2026)

Entonces para poder resolver el problema de la lentitud lo que hizo meta fue implementar una nueva arquitectura que esta lo que hacía era reemplazar el puente por un JavaScript Interface que lo que hace es que el código de JavaScript pueda tener una referencia directa en memoria de los objetos nativos de Android y con esto se logró que las llamadas entre ambos enfoques sean sincrónicas y altamente rápidas y para las vistas react no tiene que dibujar elementos web, lo que hace es llamar los componentes de Android que son subyacentes utilizando un sistema de renderizado que se llama Fabric (React Native, 2025) (React Native, 2025).

Tabla Comparativa Técnica

Criterio Técnico	Android Nativo	React Native

	(Kotlin / Java)	(JavaScript / TypeScript)
Arquitectura Central (<i>Developers, 2023</i>)	Este utiliza una ejecución directa en Android Runtime y no tiene capas que sean intermedias.	Es una arquitectura híbrida, el código JavaScript está corriendo en un motor Hermes que se comunica a través de JSI/Bridge a la capa nativa.
Mecanismo de Renderizado	Es directo a través del SDK de Android, utiliza vistas tradicionales o Jetpack Compose.	Interpreta los componentes de react y los traduce en tiempo real a componentes nativos de Android, Fabric.
Acceso a APIs del Sistema	Es síncrono, directo y nativo además de que soporta las últimas características de Android aun siendo del mismo día de lanzamiento.	Requiere un módulo de enlace que sería nativo-JS. Si es muy nuevo hay que escribir un código o esperar a que la comunidad de una solución.
Gestión de Hilos (Threading)	Nativamente es multihilo, se puede gestionar de forma manual los hilos de fondo con Coroutines.	Principalmente tiene 3 hilos de trabajo los cuales son JS lógica de negocio, UI renderizado y layout que es cálculo de sombras posiciones.
Rendimiento de Memoria	El consumo de la RAM está optimizado	Tiene mayor consumo de RAM ya que tiene que

	y tiene un impacto muy bajo en el consumo de batería.	mantener el motor de JavaScript y el recolector de basura que está corriendo en paralelo.
Tamaño del Ejecutable (APK)	Es muy ligero, puede llegar a ser de muy pocos megabytes si se optimiza.	Es más pesado porque este incluye el motor Hermes y las librerías de enlace.

Ventajas y Desventajas de React Native

Ventajas

Tiene reutilización de código interplataforma, esta es una de ventajas más grandes que tiene a nivel competitivo, porque solo se tiene que hacer un único código base de desarrollo que este va a funcionar tanto en Android como en iOS. Al tener esta ventaja que no solo va a reducir los costos iniciales a la mitad, sino que también va a unificar la parte de mantenimiento y la corrección en un solo lugar.

Flujo de trabajo eficiente ya que en el desarrollo de nativo tradicional si se hace un cambio en la interfaz aunque sea pequeño los obliga a que se tenga que compilar el proyecto y esto hace que el tiempo se extense ya que se puede llegar a esperar de segundos hasta varios minutos, en cambio con React pasa que este inyecta el código ya modificado al motor de JavaScript en tiempo de ejecución, con esto permite que se puedan ver los cambios que se están realizando en menos segundos que a como sería en Nativo, esto sin tener que perder el estado de la aplicación.

Ecosistema maduro y una bolsa grande de talento ya que al estar basado en react, JavaScript y TypeScript, las empresas no tienen por qué estar dentro de una competencia de desarrolladores que sean nativos especializados. Entonces gracias a esto se pueden del gran mercado de desarrolladores web y poder capacitarlos en el desarrollo móvil en muy poco tiempo.

Tiene una consistencia que hace que sea más fácil la lógica de negocio y los flujos sean idénticos en todas las plataformas, esto asegura que los usuarios de Android tengan la misma experiencia que tiene un usuario de iOS.

Desventajas

En computo pesado tiene una penalización de rendimiento y esto se debe a que la lógica que se tiene que procesar se hace en el hilo de JavaScript, operaciones matemáticas que son complejas, edición de videos o fotos en tiempo real y los videojuegos sufren una caída drástica de rendimiento.

Deuda de las dependencias de terceros ya que react delega muchas funciones como lo serian por ejemplo el acceso a la cámara web avanzada, biometrías y entre otras cosas a librerías que son creadas por la comunidad, entonces si alguno de estos decide abandonar el proyecto y Google actualiza Android, la aplicación puede llegar a dejar de funcionar por completo.

Retraso en la adopción de funciones nativas porque cuando Google decide lanzar una nueva característica que es exclusiva de Android en sus eventos anuales, los desarrolladores tienen que esperar a que Meta o alguien de la comunidad cree los wrappers necesarios para poder usarla en JavaScript.

Retos Encontrados en el Desarrollo Real

Depuración Compleja y Mensajes de Error Crípticos: Cuando la aplicación falla, si el error se produce en el JavaScript, la comunicación o en el código nativo de Android que es el gradle, hace que los errores sean difíciles de encontrar ya que el entorno de desarrollo muestra los fallos de compilación, pero no dan pistas claras de que línea de JavaScript los provoco.

Actualizaciones incompatibles: Realizar una actualización de react a una superior es muy difícil ya que los cambios internos que ocurren hacen que se rompa la compatibilidad con la configuración de compilación de Android y hace que se tenga que pelear bastante con las dependencias nativas incompatibles.

Desempeño de animaciones e interfaces complejas ya que al intentar crear una animación que responda rápido al usuario debe tener un gran esfuerzo de optimización. Si el hilo de JavaScript está ocupado procesando datos de la API, la animación se congela por un momento y la sensación de que la aplicación es lenta.

Análisis Crítico del Grupo

React native tiene un compromiso en arquitectura bastante bueno, pero este es condicionado. La tecnología no debe de tomarse a la ligera solo como una solución para crear aplicaciones más baratas, si no como una decisión basada en el tipo de producto.

Se identifico que, aunque React sea solo de escribir una vez y ya funciona en todos lados tiene detrás una realidad técnica más compleja porque las aplicaciones que son grandes de React Native nunca son de 100% de JavaScript entonces esto hace que se cuenta con alguien especializado o entienda muy bien el entorno de Android nativo, su sistema de permisos, manejo de hilos y el gradle que es el gestor de las dependencias.

Para las aplicaciones que su propósito es el procesamiento de datos, formularios y redes sociales, React es muy bueno o el mejor en velocidad de entrega y costo, pero si lo que se necesita es que el producto sea exprimir el hardware de animaciones fieles o uso de algoritmos locales, arquitectura limpia y entre otros aspectos, la tecnología más adecuada seria Android nativo técnicamente.

Bibliografía

- Bonaventure, O. (12 de Diciembre de 2023). Obtenido de <https://blog.logrocket.com/leveraging-react-native-jsi-enhance-speed-performance>
- Chisti, F. (2025). *React Native New Architecture 2025: Fabric & TurboModules Guide*. Obtenido de isitdev.com: <https://isitdev.com/react-native-new-architecture-fabric-turbomodules-2025-2/>
- Developers*. (8 de Mayo de 2023). Obtenido de <https://source.android.com/docs/core/runtime>
- Konicek, M. (2016). *React Native: A year in review*. Obtenido de Engineering at Meta: <https://engineering.fb.com/2016/04/13/android/react-native-a-year-in-review/>
- Liu, Y., Chen, X., Liu, P., Grundy, J., Chen, C., & Li, L. (2023). ReuNify: A Step Towards Whole Program Analysis for React Native Android Apps. *arXiv*. Obtenido de <https://arxiv.org/abs/2309.03524>
- Meta. (2025). *Introducing the React Foundation: The New Home for React & React Native*. Obtenido de Engineering at Meta: https://engineering.fb.com/2025/10/07/open-source/introducing-the-react-foundation-the-new-home-for-react-react-native/?utm_source=chatgpt.com
- Meta. (2025). *React Native*. Obtenido de Architecture Overview: https://reactnative.dev/architecture/overview?utm_source=chatgpt.com
- React Native*. (2025). Obtenido de <https://reactnative.dev/architecture/fabric-renderer>
- React Native*. (2025). Obtenido de <https://reactnative.dev/architecture/overview>
- Soni, V. (22 de Abril de 2026). *DianApps*. Obtenido de <https://dianapps.com/blog/react-native-new-architecture/>

Referencias

- Bonaventure, O. (12 de Diciembre de 2023). Obtenido de <https://blog.logrocket.com/leveraging-react-native-jsi-enhance-speed-performance>
- Chisti, F. (2025). *React Native New Architecture 2025: Fabric & TurboModules Guide*. Obtenido de isitdev.com: <https://isitdev.com/react-native-new-architecture-fabric-turbomodules-2025-2/>

Developers. (8 de Mayo de 2023). Obtenido de <https://source.android.com/docs/core/runtime>

Konicek, M. (2016). *React Native: A year in review*. Obtenido de Engineering at Meta: <https://engineering.fb.com/2016/04/13/android/react-native-a-year-in-review/>

Liu, Y., Chen, X., Liu, P., Grundy, J., Chen, C., & Li, L. (2023). ReuNify: A Step Towards Whole Program Analysis for React Native Android Apps. *arXiv*. Obtenido de <https://arxiv.org/abs/2309.03524>

Meta. (2025). *Introducing the React Foundation: The New Home for React & React Native*. Obtenido de Engineering at Meta: https://engineering.fb.com/2025/10/07/open-source/introducing-the-react-foundation-the-new-home-for-react-react-native/?utm_source=chatgpt.com

Meta. (2025). *React Native*. Obtenido de Architecture Overview: https://reactnative.dev/architecture/overview?utm_source=chatgpt.com

React Native. (2025). Obtenido de <https://reactnative.dev/architecture/fabric-renderer>

React Native. (2025). Obtenido de <https://reactnative.dev/architecture/overview>

Soni, V. (22 de Abril de 2026). *DianApps*. Obtenido de <https://dianapps.com/blog/react-native-new-architecture/>